

Лабораторная работа №18. Введение в JavaScript. Сравнение с C#

Цель работы:

- Познакомиться с языком программирования JavaScript;
- Понять его место в web-разработке;
- Сравнить базовые концепции JavaScript и C#;
- Научиться запускать JavaScript-код в браузере и через Node.js.

Краткая теория (вводная часть)

JavaScript (JS) — это интерпретируемый язык программирования, который изначально создавался для работы в браузере и управления поведением веб-страниц.

Основные особенности JavaScript:

- выполняется в **браузере** (frontend) и на **сервере** (Node.js);
- **динамическая типизация**;
- интерпретируемый язык (без компиляции в привычном смысле);
- активно работает с:
 - HTML;
 - CSS;
 - DOM.
- широко используется для:
 - интерактивных сайтов;
 - одностраничных приложений (SPA);
 - серверных API (Node.js).

JavaScript и C# — ключевые отличия

Характеристика	C#	JavaScript
Типизация	Статическая	Динамическая
Компиляция	Да	Нет (интерпретация / JIT)
Основная среда	.NET	Браузер / Node.js
ООП	Классическая	Прототипная (классы — синтаксический сахар)
Точка входа	Main()	Отсутствует

Шаг 1. Подготовка рабочего окружения

1. Откройте **Visual Studio Code**
2. Откройте встроенный терминал
3. Перейдите в каталог с документами
4. Создайте директорию для лабораторной работы:

Lab18_LearnJs_FIO (где FIO — ваша фамилия)

5. Перейдите в созданную директорию
6. Создайте структуру проекта:
 - **README.md**
 - **index.html**
 - **main.js**

JavaScript почти всегда используется во взаимодействии с HTML, поэтому файл index.html необходим.

7. Откройте созданную папку в **Visual Studio Code**

Шаг 2. Инициализация Git-репозитория

1. Инициализируйте Git-репозиторий:

git init

2. Добавьте файлы в индекс:

git add .

3. Создайте первый коммит:

git commit -m "Initial commit"

Шаг 3. Создание удалённого репозитория

1. Создайте пустой **public-репозиторий** на GitHub с именем:

Lab18_LearnJs

2. Не добавляйте:

- **README**
- **.gitignore**
- **License**

3. Привяжите локальный репозиторий к удалённому:

git branch -M main

git remote add origin <URL_репозитория>

4. Отправьте файлы:

git push -u origin main

5. Сделайте **скриншот терминала** с выполненными командами:

- **commit**
- **remote add**
- **push**

6. Создайте папку **img** и сохраните в ней изображение удачного пуша под именем:

gitPushLab18_FIO.png

7. Добавьте изменения и отправьте их:

git add .

git commit -m "Added push screenshot and project update"

git push

Шаг 4. Подключение JavaScript к HTML

Вариант 1 (рекомендуемый): В конце `<body>`

Откройте файл **index.html** и добавьте базовую HTML-структуру:

```
1 <!DOCTYPE html>
2 <html lang="ru-Ru">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width,
6       initial-scale=1.0" />
7     <title>Lab 17 - JavaScript Basics</title>
8   </head>
9   <body>
10    <h1>Learn JS</h1>
11    <script src="main.js"></script>
12  </body>
13 </html>
```

JavaScript-файл подключается с помощью тега `<script>` — аналог подключения библиотек в C# через `using`. Плюсы: сперва загружается страница, после скрипт.

Вариант 2: В `<head>` с атрибутом `<defer>`

```
<head>
<script src="main.js" defer></script>
```

Атрибут `defer` откладывает выполнение скрипта до полной загрузки страницы.

Сравнение с C#:

В WinForms/WPF вы привыкли, что код выполняется после загрузки формы. В JavaScript нужно **явно** контролировать порядок загрузки.

Шаг 5. Первый JavaScript-код

Откройте файл **main.js** и добавьте:

```
1. console.log("Hello, JavaScript!");
```

2. Откройте **index.html** в браузере
3. Откройте **DevTools** → **Console** (клавиша F12)
4. Убедитесь, что сообщение отображается

C#	JavaScript
<code>Console.WriteLine()</code>	<code>console.log()</code>
Программа запускается явно	Код выполняется при загрузке страницы

Шаг 6. Переменные и типы данных в JavaScript

6.1. Теоретическая часть

В JavaScript **нет явного указания типа переменной**, как в C#.

Тип определяется **во время выполнения программы**.

В JavaScript используются три ключевых слова для объявления переменных:

- **let** — изменяемая переменная
- **const** — константа
- **var** — устаревший способ (использовать не рекомендуется)

Сравнение с C#

C#	JavaScript
<code>int x = 10;</code>	<code>let x = 10;</code>
<code>string s = "text";</code>	<code>let s = "text";</code>
<code>bool flag = true;</code>	<code>let flag = true;</code>
Тип известен заранее	Тип определяется в runtime

6.2. Базовый пример (обязателен к переписыванию)

Задание: вручную перепишите следующий код в файл **main.js**:

```
let age = 20;
let name = "Denis";
let isStudent = true;

console.log("Name: ", name);
console.log("Age: ", age);
console.log("Is student: ", isStudent);
```

1. Откройте **index.html** в браузере.
2. Откройте **DevTools** → **Console**
3. Убедитесь, что значения корректно выводятся:

Hello, JavaScript!

Name: Denis

Age: 20

Is student: true

6.3. Изменение типа переменной

В JavaScript одна и та же переменная может менять тип.

Пример (обязателен к переписыванию):

```
let value = 10;
console.log(value);
value = "Теперь это строка";
console.log(value);
value = true;
console.log(value);
```

Да, JavaScript позволяет менять тип переменной, Но так делать НЕ НУЖНО!

Почему это плохо:

1. Ошибки в коде
2. Сложно читать код

Другой программист не поймёт, что в переменной может быть.

3. Отладка занимает больше времени

Ошибки типов находятся только во время выполнения.

Сравнение с C#:

В C# компилятор запрещает менять тип!



Позже мы изучим TypeScript, который добавляет строгую типизацию в JS.

6.4. Типы данных

В данном шаге рассмотрим основные типы данных, и некоторые особенности языка JavaScript.

1. Примитивные (primitive) типы

String (Строка)

```
let userName = "Алексей";
// шаблонная строка
console.log(`Привет, ${userName}!`);
```

Number (Число)

```
// дробное число
let price = 99.99;
// отрицательное число
let temperature = -15;
// Infinity
let infinity = 1 / 0;
// NaN (Not a Number)
let notANumber = 0 / 0;
// 0.30000000000000004 (особенность JS)
console.log(0.1 + 0.2);
```

BigInt (Большие целые числа)

```
let bigNumber = 9007199254740991n; // добавляем "n" в конце
let huge = BigInt("123456789012345678901234567890");
```

Boolean (Логический тип)

```
let isAlive = true;
let isWorking = false;
let isAdult = age ≥ 18; // результат сравнения
```

Undefined (Не определено)

```
let x; // переменная объявлена, но не присвоено значение
let y = undefined; // явное присваивание
```

Null (Пустое значение)

```
let userData = null;
```

Что такое NaN?

NaN = **Not a Number** (Не число).

Появляется, когда JS не может преобразовать строку в число:

```
console.log("Hello" - 5); // NaN
```

```
console.log(Number("Hello")); // NaN
```

```
console.log(0 / 0); // NaN
```

 **Особенность:** NaN не равен самому себе!

```
console.log(NaN === NaN); // false
```

Symbol (Символ)

2. Объектные (object) типы

```
let id = Symbol("id"); // создает уникальный идентификатор
```

Object (Объект)

```
let person = {
  name: "Станислав",
  age: 30,
  isStudent: false,
  sayHello: function () {
    console.log("Привет!");
  },
};

console.log(person.name);
```

Array (Массив)

```
let fruits = ["яблоко", "банан", "апельсин"];
let numbers = [1, 2, 3, 4, 5];
let mixed = ["текст", 42, true, null];
```

Function (Функция)

```
function sum(a, b) {
  return a + b;
}

let multiply = function(x, y) {
  return x * y;
};

console.log(sum(5, 3));
```

Date (Дата)

```
let now = new Date();
let birthday = new Date("1995-12-17");
```

6.6. Арифметические операции

<pre>let a = 10; let b = 3;</pre>	<pre>console.log(a + b); console.log(a - b); console.log(a * b); console.log(a / b);</pre>
-----------------------------------	--

Особенность JavaScript:

```
console.log(10 + "5"); // "105"
console.log("10" - 5); // 5
```

Сравнение с С#: В С# такое невозможно без приведения типов. В JS происходит неявное преобразование

6.5. Константы (const)

```
> const PI = 3.14;
  console.log(PI);
  PI = 3.1415; // Ошибка
3.14
```

✖ ▶ Uncaught TypeError: Assignment to constant variable.
at <anonymous>:3:4

const запрещает переназначение, но НЕ запрещает изменение содержимого!

Массивы:

```
const numbersArray = [1, 2, 3];
// Можно изменять элементы:
numbersArray[0] = 10;
console.log(numbersArray); // [10, 2, 3]
// Нельзя переназначить:
numbersArray = [5, 6, 7]; // Ошибка!
```

Объекты:

```
const persons = { name: "Denis", age: 18 };
// Можно изменять свойства:
persons.age = 50;
persons.city = "Volgograd";
console.log(persons); // { name: "Denis", age: 50, city: "Volgograd" }
// Нельзя переназначить:
person = { name: "Stas" }; // Ошибка!
```

Сравнение с С#:

В С# `const` работает только с примитивами. Для объектов используется `readonly`.

Практическое задание

1. Сделайте скриншоты файла **main.js** и сохраните в папку **img** с именем:

step6_variablesLab18_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 6: variables and data types in JavaScript"

git push

Шаг 7. Проверка типов

7.1. Краткое описание

typeof — оператор для определения типа данных в JavaScript.

Особенности:

- `typeof null` возвращает `"object"` (историческая особенность)
- массивы тоже `"object"`
- функции имеют тип `"function"`

7.2. Пример кода

```
console.log(typeof "текст"); // "string"
console.log(typeof 42); // "number"
console.log(typeof true); // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof null); // !!! "object"
console.log(typeof {}); // "object"
console.log(typeof []); // "object"
console.log(typeof function () {}); // "function"
```

Особенность: `typeof null`

Это историческая ошибка JavaScript, которую нельзя исправить (сломается много кода). `null` — это примитивный тип, но `typeof` ошибочно возвращает `"object"`.

Как правильно проверить на `null`:

```
let numberX = null;
console.log(numberX === null); // true
```



В C# такой проблемы нет: `null` корректно определяется как отсутствие значения.

Практическое задание

Задание:

1. Создайте переменную `newPrice`
2. Присвойте ей число
3. Выведите значение и тип
4. Присвойте строку
5. Снова выведите значение и тип

7.3. Практическое задание

1. Сделайте скриншот файла **main.js** и сохраните в папку **img** с именем:

step7_typeofLab18_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 7: checking data types with typeof"

git push

Шаг 8. Явные и неявные преобразования

8.1. Краткое описание

- **Явное преобразование** — когда программист сам преобразует тип:

Number(), String(), Boolean(), parseInt(), parseFloat(), toString(), +

- **Неявное преобразование** — когда JS сам пытается привести тип:

"5" + 3 → "53" (конкатенация), "5" - 3 → 2 (число)

8.2. Примеры кода

Явное преобразование типов:

```
// В строку
let num = 42;
let str = String(num); // "42"
let str2 = num.toString(); // "42"
let str3 = "" + num; // "42"
```

```
// В число
let strNum = "123";
let int = Number(strNum); // 123
let int2 = parseInt("42.5"); // 42
let float = parseFloat("3.14"); // 3.14
let int3 = + "99"; // 99
```

```
// В булево значение
let bool1 = Boolean(1); // true
let bool2 = !!1; // true
let bool3 = Boolean(0); // false
let bool4 = Boolean(""); // false
```

Неявное преобразование (coercion):

```
console.log("5" + 3); // "53" (конкатенация)
console.log("5" - 3); // 2 (преобразование в число)
console.log("5" * "2"); // 10
console.log(true + 1); // 2
console.log(false + 1); // 1
console.log(null + 1); // 1
console.log(undefined + 1); // NaN
```

Почему так работает?

Правило 1: Оператор `+` с строкой → конкатенация

`console.log("5" + 3); // "53"` (число превратилось в строку)

Правило 2: Операторы `-`, `*`, `/` → преобразование к числу

`console.log("5" - 3); // 2` (строка стала числом)

`console.log("10" * 2); // 20`

⚠ Ловушка:

`console.log("5" + 3 + 2); // "532"` (строка + число + число)

Вывод: Всегда явно преобразуйте типы, чтобы избежать ошибок!

Сравнение с C#:

В C# такое невозможно! Он выдаст ошибку компиляции.

💡 JavaScript слишком гибкий — это и плюс, и минус.

8.3. Практическое задание

1. Сделайте скриншоты файла **main.js** и сохраните в папку **img** с именем:

step8_typeConversionLab18_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 8: explicit and implicit type conversion"

git push

Шаг 9. Строгое и нестрогое сравнение

9.1. Краткое описание

- `==` — нестрогое сравнение (JS может привести типы)
- `===` — строгое сравнение (без преобразования типов)

```
console.log(5 == "5"); // true (нестрогое, с преобразованием)
console.log(5 === "5"); // false (строгое, без преобразования)
console.log(0 == false); // true
console.log(0 === false); // false
console.log(null == undefined); // true
console.log(null === undefined); // false
```

⚠ Объекты сравниваются по **ссылке**, а не по содержимому:

```
let obj1 = { name: "John" };
let obj2 = { name: "John" };

console.log(obj1 == obj2); // false
console.log(obj1 === obj2); // false
```

Почему? Это разные объекты в памяти!

```
let obj3 = obj1; // obj3 ссылается на тот же объект
console.log(obj1 === obj3); // true
```

То же с массивами:

```
let arr1 = [1, 2, 3];
let arr2 = [1, 2, 3];
console.log(arr1 === arr2); // false (разные массивы)
```

Сравнение с C#:

В C# поведение аналогично — ссылочные типы сравниваются по ссылке.

💡 **Вывод:** `===` для объектов проверяет, **один ли это объект в памяти**.

Практическое задание

1. Сделайте скриншоты файла **main.js** и сохраните в папку **img** с именем:

step9_strictLab18_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 9: strict and loose comparison"

git push

Шаг 10. Работа с консолью браузера

10.1. Теоретическая часть

Браузерная консоль — быстрый способ проверить JavaScript-код без файлов.

Плюсы:

- мгновенный результат;
- удобно тестировать небольшие выражения;
- не нужно создавать проект.

Минусы:

- нет возможности сохранять проект;
- неудобно для больших скриптов;
- сложно работать с файлами и модулями.

10.2. Практический пример

Откройте **DevTools** → **Console** в браузере и введите:

```
> let a1 = 5;
   let b1 = 3;
   console.log("Сумма:", a1 + b1);
   console.log("Разность:", a1 - b1);
   console.log("Произведение:", a1 * b1);
   console.log("Деление:", a1 / b1);
```

Другие методы console (помимо log):

```
console.log("Обычное сообщение");
console.warn("Предупреждение!");
console.error("Ошибка!");
console.info("Информация");
// Таблица (удобно для массивов/объектов)
let users = [
  { name: "John", age: 30 },
  { name: "Jane", age: 25 },
];
console.table(users);
```

Очистка консоли:

```
console.clear();
```

10.3. Практическое задание №1

В консоли создайте переменные:

```
let x1 = 10;
```

```
let y1 = 2;
```

Выведите:

```
x1 + y1
```

```
x1 - y1
```

x1 * y1

x1 / y1

Попробуйте поменять **x1** на строку **"10"** и снова выполнить операции

Практическое задание

1. Сделайте скриншоты файла **main.js** и сохраните в папку **img** с именем:

step10_consoleLab18_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 10: browser console basics and simple math"

git push

Шаг 11. Установка Node.js и запуск скрипта

11.1. Теоретическая часть

Node.js позволяет запускать JavaScript **вне браузера**, на компьютере.

Зачем нужен Node.js?

1. Разработка без браузера

Не нужно постоянно обновлять страницу в браузере.

2. Серверная разработка

Node.js позволяет создавать серверы (backend).

3. Работа с файлами

В браузере нельзя читать/писать файлы на компьютере.

В Node.js — можно.

4. Инструменты разработки

Все современные инструменты работают на Node.js:

- **React** — запускается через Node
- **Webpack** — сборщик проектов
- **npm** — менеджер пакетов

Сравнение с C#:

Node.js для JavaScript = .NET Runtime для C#.

Без .NET вы не запустите C# приложение.

Без Node.js вы запустите JS только в браузере.

Вывод: Node.js расширяет возможности JavaScript **далеко** за пределы браузера.

11.2. Установка Node.js

1. Перейдите на официальный сайт: <https://nodejs.org>
2. Скачайте LTS версию и установите
3. Проверьте установку в терминале:

node -v

npm -v

```
denis@denis-Desktop ~> node -v
v24.12.0
denis@denis-Desktop ~> npm -v
11.6.2
```

11.3. Запуск скрипта через Node.js

node main.js

```
denis@denis-Desktop ~/D/I/Lab9_LearnJs_Leontev (main)> node main.js
Hello, JavaScript!
Name: Denis
Age: 20
Is student: true
```

Должен появиться тот же вывод, что в консоли браузера

11.4. Практическое задание №1

- Создайте переменные **a2** и **b2**
- Сложите, умножьте, выведите результат через **console.log()**
- Запустите скрипт через Node

Практическое задание

1. Сделайте скриншот терминала с запуском **Node** и сохраните в папку **img** с именем:

step11_nodeLab18_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 11: Node.js installed and main.js executed"

git push

Шаг 12. Условные операторы и логика в JavaScript

12.1. Теоретическая часть

Условные конструкции в JavaScript по синтаксису **очень похожи на C#**, но есть важные отличия:

- нет строгой типизации;
- условия могут работать с любыми типами;
- есть строгое и нестрогое сравнение.

Таблица всех операторов сравнения:

Оператор	Название	Приводит типы?	Пример
==	Равно	Да	5 == "5" → true
===	Строго равно	Нет	5 === "5" → false
!=	Не равно	Да	5 != "5" → false
!==	Строго не равно	Нет	5 !== "5" → true

12.2. Оператор if / else

Базовый пример (обязателен к переписыванию):

```
let yourAge = 18;

if (yourAge >= 18) {
  console.log("Доступ разрешён");
} else {
  console.log("Доступ запрещён");
}
```

12.3. Практическое задание №1

Задание:

1. Создайте переменную **temperature**
2. Если температура **меньше 0** — вывести "Холодно"
3. Если **от 0 до 20** — "Прохладно"
4. Если **больше 20** — "Тепло"

Использовать if / else if / else

12.4. Логические операторы

В **JavaScript** используются те же логические операторы, что и в **C#**:

- **&&** — И
- **!** — НЕ

Пример:

```
if (isStudent && age < 25) {
  console.log("Доступна студенческая скидка");
}
```


12.5. Практическое задание №2

Задание:

1. Создайте переменные:
 - `isLoggedIn`
 - `isAdmin`
2. Если пользователь авторизован **и** администратор — вывести "Полный доступ"
3. Если авторизован, но не администратор — "Ограниченный доступ"
4. Если не авторизован — "Доступ запрещён"

12.6. Практическое задание №3

Задание:

1. Создайте переменные:
 - `a3 = 10`
 - `b3 = "10"`
2. Сравните их с помощью:
 - `==`
 - `===`
3. Выведите **результаты** в **консоль**
4. Сделайте вывод, в чём разница

12.7. Тернарный оператор

Пример:

```
let message = age >= 18 ? "Совершеннолетний" : "Несовершеннолетний";  
console.log(message);
```

Используется для коротких условий, но не для сложной логики.

12.8. Конструкция switch / case

В JavaScript есть оператор **switch**, синтаксис почти такой же, как в C#.

Пример:

```
let day = 3;  
switch (day) {  
  case 1: console.log("Понедельник"); break;  
  case 2: console.log("Вторник"); break;  
  case 3: console.log("Среда"); break;  
  default: console.log("Неизвестный день");  
}
```

Если мы забудем блок **break**, то код продолжит выполнение! Это называется "**fall-through**" (проваливание).

switch / case в **JavaScript** используется реже, чем в **C#**.

12.9. Практическое задание №4 (обзорное)

Задание:

1. Создайте переменную **monthNumber** (от 1 до 12)
2. Используя **switch**, выведите название месяца
3. Добавьте **default** на случай неверного значения

Практическое задание

1. Сделайте скриншот файла **main.js** и сохраните в папку **img** с именем:

step12_conditionsLab18_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 12: conditional statements in JavaScript"

git push

Самостоятельное задание

Цель задания: Создать подробное и структурированное описание лабораторной работы №18 в файле **README.md**, используя **Markdown**.

Что должно быть в README.md

1. Заголовок

2. Основная информация

****ФИО:** Иванова Иван Иванович**

****Группа:** ИСП-231**

****Дата:** 18.02.2026**

3. Краткое описание работы

Описание (что изучили)

4. Структура проекта

Структура проекта

Итоговая таблица: JavaScript vs C#

Аспект	C#	JavaScript
Типизация	Статическая	Динамическая
Компиляция	Да (в IL-код)	Нет (интерпретация/JIT)
Точка входа	Main()	Отсутствует
Переменные	int x = 5;	let x = 5;
Константы	const int X = 5;	const X = 5;
Вывод в консоль	Console.WriteLine()	console.log()
Проверка типа	Во время компиляции	typeof
Сравнение	== (по значению)	=== (строгое)
Условия	if/else/switch	if/else/switch + тернарный
Циклы	for/while/do-while/foreach	for/while/do-while/for...of
Функции	Методы с типами	Функции без типов
Массивы	int[] arr = new int[5];	let arr = [];
Объекты	Требуют класс	Можно создать литералом
ООП	Классическое	Прототипное
Null	null	null и undefined
Строгость	Высокая (компилятор)	Низкая (нужна дисциплина)

Главные выводы:

1. **JavaScript гибче C#** — можно менять типы переменных
2. **JavaScript проще синтаксически** — меньше церемоний
3. **JavaScript опаснее** — ошибки находятся во время выполнения
4. **C# безопаснее** — компилятор не даст допустить многие ошибки

После сделайте в терминале:

git add .

git commit -m "Added complete README for Lab18"

git push